

- **Self-service catalogue:** Allows developers to provision resources without manual intervention from IT admins.

Enterprise container management layer: Containerising the solution makes the services cloud-agnostic and platform-agnostic. Containers provide a platform to run services that are easily portable, elastically scalable, multi-region deployable and resilient, based on Kubernetes.

Examples include Kubernetes distributions (e.g., OKD, RKE2), specialised GitOps and CI/CD pipelines (e.g., ArgoCD, Jenkins), and an observability stack (e.g., Prometheus, Grafana).

Hybrid infrastructure and edge environments: This is a foundation layer that has physical or virtualised hardware on which everything runs. By utilising open source technologies like KVM-based hypervisors for private clouds and edge computing solutions, this layer creates a consistent abstraction across public clouds, private clouds, on-premises data centres, SaaS and the network

edge. Examples are KubeEdge and OpenYurt.

- **Edge tier:** Localised nodes for <10ms latency processing.
- **Private sovereign cloud:** Secure, on-premises data centres that keep sensitive information within national or corporate borders.
- **Public cloud (Hyperscalers):** Used for massive scaling, burst compute, and heavy AI model training.
- **Infrastructure as Code (OpenTofu):** Ensures the entire foundation is scripted and reproducible.

Unified core services: This layer comprises the standards and processes that must be applied consistently across every single level. It prevents vendors locking in and provides a ‘common language’ for the entire platform. By using open standards (like Terraform for infrastructure or Prometheus for monitoring), it ensures that we can move a service from public cloud to an on-premises private cloud without having to rewrite the deployment scripts or change how the app is monitored.

Unified observability: This provides real-time metrics, logs, and traces from

the hardware right up to the user; e.g., Open Telemetry.

Security, compliance and regulatory trust: This layer acts as automated guardrails. It ensures that data sovereignty, zero-trust identity (Keycloak), and regulatory requirements are met at every layer.

DevSecOps and platform

operations: This layer automates the security and maintenance of the platform throughout the software lifecycle.

A health insurance claims management platform moving to a hybrid cloud model: An example

Let’s consider a health insurance company modernising its claims management platform using hybrid cloud architecture to improve adjudication speed, regulatory compliance, and operational resilience.

The platform processes millions of claims daily. The solution is based on a hybrid cloud model that is centered around AI and addresses the following:

Edge computing: The solution adopts AI-powered claims triage models

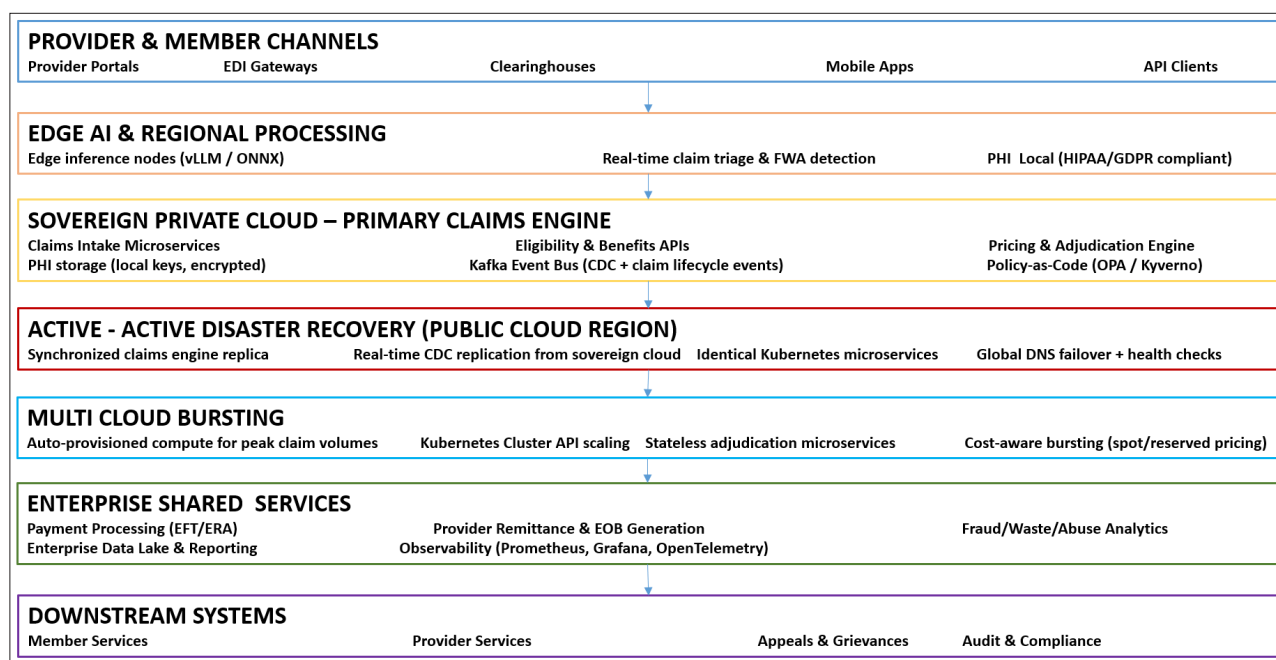


Figure 2: Hybrid cloud claims management architecture